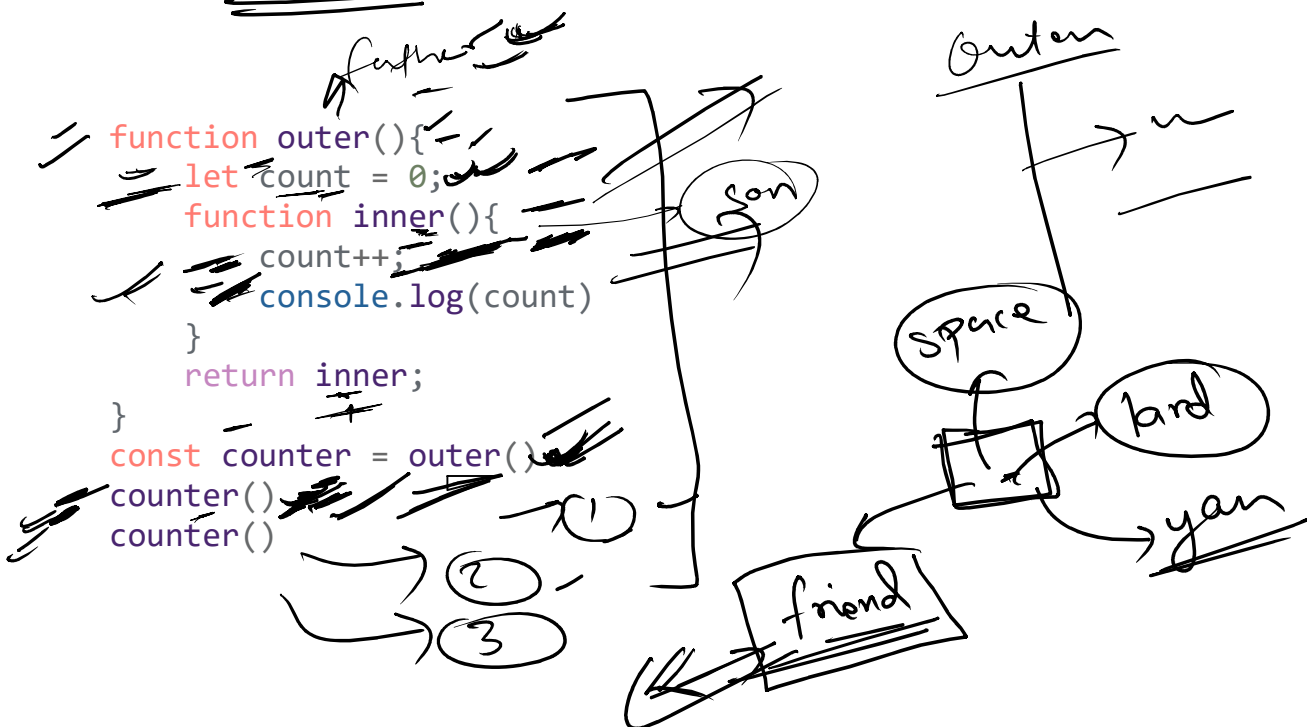


Closures and Function currying

Closures



A closure happens when an inner function remembers and can access variable from its outer function even after outer function has finished executing.

Why closures are useful

- Data privacy
- counters
- callbacks
- event handlers

Real life example

```
function createBankAccount(){
  let balance = 10000;
  return {
    getBalance : function (){
      return balance
    },
    deposit : function (amount){
      balance += amount
    }
  }
}

const account = createBankAccount();
console.log(account.getBalance())
account.deposit(500)
console.log(account.getBalance())
```

Function Currying

→ Currying means converting a function that takes multiple arguments in a sequence

of ~~fun~~ functions where each function takes one arguments.

```
// function currying  
// normal function  
// function add(a,b){  
//     return a+b  
// }
```

```
// currying function  
function add(a){  
    return function (b){  
        return a+b  
    }  
}  
console.log(add(5)(5))
```

add(10) ↓
↑ a — b

add(10)(5)
↓ a ↓ b
a+b
15

~~add(10)(5)~~

```
function multiply (a){  
    return function (b){  
        return a * b  
    }  
}  
const double = multiply(2)  
console.log(double(5))  
const triple = multiply(3)  
console.log(triple(5))
```